Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Кафедра програмування

Звіт

Про виконану роботу з курсу Програмної інженерії

Проєкт «PetSearchHome»

Заняття № 6

Львів – 2026

Тема: Реалізація use-case в рамках MVC

Завдання:

- Додати для кожного проекту TreatWarningsAsErrors і виправити помилки.
- Порефакторити контролери.
- Забрати try-catch з сервісів, обробляти винятки глобально
- Використати Result патерн для повернення результатів виконання сервісів.
- Написання контролерів (controller) для обробки запиту користувача, що відповідає use-case
- Написання класів сутностей та сервісів (model) для виконання бізнес-логіки
    - Дотримання правил написання "чистого" коду
    - Логування дій користувача та помилок
- Написання класів (репозиторіїв) для отримання чи зміни даних в базі даних
- Створення шаблонів відображень (view) для виконання use-case користувачем з використанням Razor view engine у ASP .NET Core MVC
- Написання юніт-тестів з використанням xUnit або NUnit

Для виконання поставлених завдань роботу було розподілено між учасниками команди таким чином:

- **Лук'янчук Денис** – додав для кожного проекту TreatWarningsAsErrors і виправити помилки, реалізував use-case <u>додавання до улюблених</u>, написання звіту про виконану роботу.

- **Денисюк Денис** – написав репозиторії та сервіси (models), доопрацював БД, реалізував use-case <u>пошук тварин.</u>

- **Патрило Павло** – написав контролери (що відповідає use-case), реалізував use-case <u>скарга на оголошення.</u>

- **Бойко Максим** – порефакторив контролери, забрав try-catch з сервісів, написання тестів, реалізував use-case <u>перегляд деталей оголошення.</u>

- **Левус Любомир** – використав Result патерн для повернення результатів виконання сервісів, написання тестів, реалізував use-case <u>фільтрація оголошень</u>.

1. Архітектура

Основні зміни:

- У всіх проектах додано TreatWarningsAsErrors, що дозволило виявити та виправити потенційні проблеми на етапі компіляції
- Контролери були спрощені — тепер вони лише викликають сервіси
- Видалено try-catch із сервісів

Для обробки помилок використано глобальний обробник винятків (middleware), що забезпечує:

- централізовану обробку помилок
- логування
- повернення коректних відповідей клієнту

2. Використання Result патерну

У сервісах було впроваджено Result патерн для повернення результатів виконання. Це дозволило:

- уникнути використання винятків у бізнес-логіці
- явно повертати результат (успіх або помилка)
- спростити обробку результатів у контролерах

Контролери перевіряють результат і формують відповідь залежно від статусу виконання.

3. Реалізація use-case

Було реалізовано наступні use-case:

3.1. Додавання до улюблених

Користувач може додати оголошення до списку улюблених. Дані зберігаються в базі даних та прив'язуються до користувача.

3.2. Пошук тварин

Реалізовано пошук за різними параметрами (тип, порода, вік тощо). Використано сервіси та репозиторії для отримання даних.

3.3. Скарга на оголошення

Користувач може залишити скаргу на оголошення. Інформація зберігається в базі для подальшої модерації.

3.4. Перегляд деталей оголошення

Користувач може переглянути детальну інформацію про тварину, включаючи опис, фото та контакти.

3.5. Фільтрація оголошень

Реалізовано можливість фільтрації за різними критеріями (місто, тип, статус тощо).

4. Логування

Було реалізовано логування:

- дій користувача
- помилок
- винятків

Логування інтегровано з глобальною обробкою помилок.

5. Репозиторії

Реалізовано репозиторії для:

- отримання даних
- зміни даних

- взаємодії з базою даних

Використано підхід через інтерфейси для забезпечення гнучкості та тестованості.

6. Представлення (View)

Створено Razor Views для:

- пошуку
- перегляду оголошень
- взаємодії користувача з системою

Інтерфейс забезпечує зручну роботу з реалізованими use-case.

7. Тестування

Було написано юніт-тести з використанням xUnit / NUnit.

Протестовано:

- сервіси
- бізнес-логіку use-case

Перевірено:

- позитивні сценарії (успішне виконання)
- негативні сценарії (помилки, некоректні дані)

Дотримано best practices:

- ізоляція тестів
- використання моків
- читабельність коду

Висновки

У ході виконання роботи було вдосконалено архітектуру проєкту, проведено рефакторинг контролерів та сервісів із впровадженням глобальної обробки винятків і Result патерну, що дозволило чітко розділити бізнес-логіку та представлення, підвищити читабельність і масштабованість коду, забезпечити централізоване логування дій користувача та помилок, а також реалізувати юніт-тестування для перевірки правильності роботи всіх use-case у різних сценаріях, що забезпечило підвищення якості, надійності та тестованості системи.